

Git and Github

Git vs Github :

- git is a software that enables the system to understand a certain set of commands that does various things related to files and documents.
- github is a service provider that allows us to save the files on a cloud . There are many other software that provides the same service but Github is the most famous one .
- repo is a collection of file that are being tracked by git . Tracking waisi hi hai jaise game khelte samay har stage ko save karte hai taki pura wapass ne na khelna pade waise hi git ko use kar k hum checkpoints add karte hai taki dobara se pura software design na karna pade
- commit is just another word for checkpoint . When we commit something we are adding the checkpoint the reason of adding the checkpoint is discussed above

flow of git

write---->add---->commit

git init

it is done only one time in a project

- we are always on any one of the branches
- git add . -----> adds all the files in the current directory to the staging area

Gitignore recommendations

<https://www.toptal.com/developers/gitignore>

- in the world of git every commit is dependent on previous commit except the first commit
- .gitignore command is used to mention the files that we don't want to track like .env files that has sensitive info that can't be shared we can add any folder to exclude it from being tracked
- gitignore recommendations gives us a list of files that one should ignore from being tracked and we can have a list of such files by mentioning the tech we are working on refer to the website provided

- git config commands enables us to set up our personal credentials so that git can know who is committing the files
- git status gives us the list of files that are not being tracked by the git along with the current branch
- git commit -m"message" is used to ready the files to be pushed to the cloud
- git add . -----> adds all the files present
- git add textone.txt texttwo.txt-----> adds only the mentioned files
- git log is used to see the commit history of the repo
- ls -la command sari files and folders ko detail me dikhata hai
- when we change the branch of the git and start working on that then the head points to the latest change that has been made
- git log --oneline -----> ye command hume batata hai ki branch k bare me
- git branch hume batata hai ki hum kis branch pr at the moment hai
- touch folder_name ----->folder_name nam ka ek naya file banata hai current directory me
- git merge bug_fix -----> agar hum currently main branch pr hai to ye command main branch me bug_fix branch ko merge kar dega
- git checkout branch-name -----> branch switch karta hai
- git branch-----> hume list deta hai sare branches ki aur usme highlighted branch hi current branch hoti hai
- git branch -d branch-name -----> ye given branch-name ko delete kar deta hai
- git diff --staged -----> ye command hume file me hue changes k bare me batata hai
- git switch -c bugfix-----> is command se 2 chize hoti hai 1 : nyi branch bugfix nam ki banti hai 2: git us branch me apne ap switch ho jata hai
- git switch branch-name -----> ye git ko di gyi branch me switch kr deta hai
- agar hum ek aisi file pr kam kr rahe hai jo 2 branch me common hai to hum ek branch se dusri branch me shift nhi jr sakte hai bina data loss hue jo changes humne kiye the is case me hum git stash command ka use karte hai jo changes ko ek temporary shelf me rakh deta hai phir jab hum branch switch kr lete hai to git stash pop command ka use kr k shelf wale data ko wapas la sakte hai
- git stash list----->command tumhare saved stashes ki list dikhata hai.
- git reflog ----->Git ka **recovery history** command hai.
Ye batata hai ki **HEAD aur branches ne recently kin commits par move kiya hai
- git rebase main-----> tumhari branch ko main branch ke upar "seedha line me" la deta hai. (see canvas for diagram)
- git commit -am "message"-----> this command adds and commits the file in one command
- rebase me conflicts same merging ki tarah hi resolve ho jate hai

rebase command ko main/master branch pr kabhi run nhi kiya jata it can only be run on side branch